SESSION DE RATTRAPAGE – JEE1 ET JEE2

Code source complet

Backend Spring Boot + Frontend React (tous les fichiers)

Clavstel El Schaddai BOYAOU

Classe : M1 GELO

TP2 : Spring Boot + React + MongoDB

Année universitaire 2025-2026

Sommaire des fichiers

N°	Fichier	Lignes
1	README.md	25
2	backend/pom.xml	42
3	backend/src/main/resources/application.properties	15
4	backend/src/main/java/com/facturation/FacturationApplication.java	11
5	backend/src/main/java/com/facturation/model/Client.java	34
6	backend/src/main/java/com/facturation/model/Facture.java	34
7	backend/src/main/java/com/facturation/model/FactureView.java	8
8	backend/src/main/java/com/facturation/model/Gerant.java	20
9	backend/src/main/java/com/facturation/repository/ClientRepository.java	13
10	backend/src/main/java/com/facturation/repository/FactureRepository.java	12
11	backend/src/main/java/com/facturation/repository/GerantRepository.java	10
12	backend/src/main/java/com/facturation/security/JwtFilter.java	43
13	backend/src/main/java/com/facturation/security/JwtService.java	42
14	backend/src/main/java/com/facturation/config/DataInitializer.java	41
15	backend/src/main/java/com/facturation/config/GlobalExceptionHandler.java	22
16	backend/src/main/java/com/facturation/config/SecurityConfig.java	60
17	backend/src/main/java/com/facturation/controller/AuthController.java	43
18	backend/src/main/java/com/facturation/controller/ClientController.java	84
19	backend/src/main/java/com/facturation/controller/FactureController.java	99
20	frontend/package.json	20
21	frontend/vite.config.js	7
22	frontend/index.html	12
23	frontend/.env	1
24	frontend/src/main.jsx	13
25	frontend/src/api.js	59
26	frontend/src/App.jsx	34
27	frontend/src/Layout.jsx	35
28	frontend/src/pages/LoginPage.jsx	42
29	frontend/src/pages/ClientsPage.jsx	98
30	frontend/src/pages/FacturesPage.jsx	109
31	frontend/src/pages/MesFacturesPage.jsx	40
32	frontend/src/styles.css	41

1. README.md (projet)

```
# TP2 JEE - Application de facturation (Spring Boot + React + MongoDB)

Auteur : Clavstel El Schaddai BOYAOU - M1 GELO

## Prerequis
- Java 17+ et Maven 3.9+
- Node.js 18+ et npm
- MongoDB 6/7 en local (port 27017)

## Lancement
1. Demarrer MongoDB (`mongod` ou service MongoDB).
2. Backend :  cd backend && mvn spring-boot:run      -> http://localhost:8080
3. Frontend : cd frontend && npm install && npm run dev -> http://localhost:5173

## Comptes de demonstration (crees au premier demarrage)
| Role   | Login  | Mot de passe |
|--------|--------|--------------|
| Gerant | admin  | admin123     |
| Client | moussa | client123    |
| Client | awa    | client123    |
| Client | cheikh | client123    |

## API REST
POST /api/auth/login | GET/POST /api/clients | GET/PUT/DELETE /api/clients/{id}
GET/POST /api/factures | GET/PUT/DELETE /api/factures/{id} | GET /api/factures/mes-factures
```

2. backend/pom.xml (backend)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
        <relativePath/>
    </parent>
    <groupId>com.facturation</groupId>
    <artifactId>facturation-api</artifactId>
    <version>1.0.0</version>
    <name>facturation-api</name>
    <description>TP2 JEE - API de facturation (Spring Boot + MongoDB)</description>
    <properties>
        <java.version>17</java.version>
        <jjwt.version>0.12.5</jjwt.version>
    </properties>
    <dependencies>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-mongodb</artifactId></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-security</artifactId></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-validation</artifactId></dependency>
        <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>${jjwt.version}</version></dependency>
        <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId><version>${jjwt.version}</version><scope>runtime</scope></dependency>
        <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId><version>${jjwt.version}</version><scope>runtime</scope></dependency>
        <dependency><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId><optional>true</optional></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope></dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
```

```
                    <excludes><exclude><groupId>org.projectlombok</groupId><artifactId>lombok</artif
        actId></exclude></excludes>
                </configuration>
            </plugin>
          </plugins>
      </build>
  </project>
```

3. backend/src/main/resources/application.properties (backend)

```
spring.application.name=facturation-api
server.port=8080

# --- MongoDB (remplace MySQL du TP1) ---
spring.data.mongodb.uri=mongodb://localhost:27017/facturation_db

# --- JWT (secret de 32 caracteres minimum) ---
app.jwt.secret=ChangeMoiParUnSecretTresLongEtAleatoire2026!
app.jwt.expiration-ms=7200000

# --- Origine autorisee du frontend React (Vite) ---
app.cors.allowed-origin=http://localhost:5173

# --- Messages d'erreur renvoyes au frontend ---
server.error.include-message=always
```

4. backend/src/main/java/com/facturation/FacturationApplication.java (backend)

```
package com.facturation;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class FacturationApplication {
    public static void main(String[] args) {
        SpringApplication.run(FacturationApplication.class, args);
    }
}
```

5. backend/src/main/java/com/facturation/model/Client.java (backend)

```
package com.facturation.model;

import com.fasterxml.jackson.annotation.JsonProperty;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

/**
 * Client(Idclient, nom, tel) + login/password : necessaires pour que
 * le client puisse se connecter et consulter ses factures (TP2).
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "clients")
public class Client {
    @Id
    private String id;

    @NotBlank(message = "Le nom est obligatoire")
    private String nom;

    private String tel;

    @NotBlank(message = "Le login est obligatoire")
    private String login;

```

```
    /** Jamais renvoye au frontend (ecriture seule). */
    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    private String password;
}
```

6. backend/src/main/java/com/facturation/model/Facture.java (backend)

```
package com.facturation.model;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDate;

/** Facture(Idfact, montant, objet, IdClient) + date d'emission. */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "factures")
public class Facture {
    @Id
    private String id;

    @NotNull(message = "Le montant est obligatoire")
    @Positive(message = "Le montant doit etre positif")
    private Double montant;

    @NotBlank(message = "L'objet est obligatoire")
    private String objet;

    @NotBlank(message = "Le client est obligatoire")
    private String idClient;

    private LocalDate date;
}
```

7. backend/src/main/java/com/facturation/model/FactureView.java (backend)

```
package com.facturation.model;

import java.time.LocalDate;

/** Facture enrichie du nom du client, renvoyee au frontend. */
public record FactureView(String id, Double montant, String objet,
                          String idClient, String clientNom, LocalDate date) {
}
```

8. backend/src/main/java/com/facturation/model/Gerant.java (backend)

```
package com.facturation.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

/** Gerant(Id, nom, login, password) - le mot de passe est stocke chiffre (BCrypt). */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "gerants")
public class Gerant {
    @Id
    private String id;
    private String nom;
    private String login;
```

```
19      private String password;
20  }
```

9. backend/src/main/java/com/facturation/repository/ClientRepository.java (backend)

```
 1  package com.facturation.repository;
 2
 3  import com.facturation.model.Client;
 4  import org.springframework.data.mongodb.repository.MongoRepository;
 5
 6  import java.util.List;
 7  import java.util.Optional;
 8
 9  public interface ClientRepository extends MongoRepository<Client, String> {
10      Optional<Client> findByLogin(String login);
11
12      List<Client> findByNomContainingIgnoreCaseOrTelContaining(String nom, String tel);
13  }
```

10. backend/src/main/java/com/facturation/repository/FactureRepository.java (backend)

```
 1  package com.facturation.repository;
 2
 3  import com.facturation.model.Facture;
 4  import org.springframework.data.mongodb.repository.MongoRepository;
 5
 6  import java.util.List;
 7
 8  public interface FactureRepository extends MongoRepository<Facture, String> {
 9      List<Facture> findByIdClient(String idClient);
10
11      void deleteByIdClient(String idClient);
12  }
```

11. backend/src/main/java/com/facturation/repository/GerantRepository.java (backend)

```
 1  package com.facturation.repository;
 2
 3  import com.facturation.model.Gerant;
 4  import org.springframework.data.mongodb.repository.MongoRepository;
 5
 6  import java.util.Optional;
 7
 8  public interface GerantRepository extends MongoRepository<Gerant, String> {
 9      Optional<Gerant> findByLogin(String login);
10  }
```

12. backend/src/main/java/com/facturation/security/JwtFilter.java (backend)

```
 1  package com.facturation.security;
 2
 3  import io.jsonwebtoken.Claims;
 4  import io.jsonwebtoken.JwtException;
 5  import jakarta.servlet.FilterChain;
 6  import jakarta.servlet.ServletException;
 7  import jakarta.servlet.http.HttpServletRequest;
 8  import jakarta.servlet.http.HttpServletResponse;
 9  import lombok.RequiredArgsConstructor;
10  import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
11  import org.springframework.security.core.authority.SimpleGrantedAuthority;
12  import org.springframework.security.core.context.SecurityContextHolder;
13  import org.springframework.stereotype.Component;
14  import org.springframework.web.filter.OncePerRequestFilter;
15
16  import java.io.IOException;
17  import java.util.List;
18
19  /** Lit l'en-tete "Authorization: Bearer <jwt>" et authentifie la requete. */
```

```java
@Component
@RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain chain) throws ServletException, IOException {
        String header = request.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                Claims claims = jwtService.parse(header.substring(7));
                var auth = new UsernamePasswordAuthenticationToken(
                        claims.get("uid", String.class), null,
                        List.of(new SimpleGrantedAuthority("ROLE_" + claims.get("role",
  String.class))));
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException | IllegalArgumentException e) {
                SecurityContextHolder.clearContext(); // jeton invalide ou expire
            }
        }
        chain.doFilter(request, response);
    }
}
```

13. backend/src/main/java/com/facturation/security/JwtService.java

(backend)

```java
package com.facturation.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

/** Creation et verification des jetons JWT. */
@Service
public class JwtService {

    @Value("${app.jwt.secret}")
    private String secret;

    @Value("${app.jwt.expiration-ms}")
    private long expirationMs;

    private SecretKey key() {
        return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    public String generate(String login, String role, String userId) {
        Date now = new Date();
        return Jwts.builder()
                .subject(login)
                .claim("role", role)
                .claim("uid", userId)
                .issuedAt(now)
                .expiration(new Date(now.getTime() + expirationMs))
                .signWith(key())
                .compact();
    }

    public Claims parse(String token) {
        return Jwts.parser().verifyWith(key()).build().parseSignedClaims(token).getPayload();
    }
}
```

14. backend/src/main/java/com/facturation/config/DataInitializer.java

(backend)

```java
package com.facturation.config;

import com.facturation.model.Client;
import com.facturation.model.Facture;
import com.facturation.model.Gerant;
import com.facturation.repository.ClientRepository;
import com.facturation.repository.FactureRepository;
import com.facturation.repository.GerantRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

import java.time.LocalDate;

/** Donnees de demonstration creees au premier demarrage (base vide). */
@Component
@RequiredArgsConstructor
public class DataInitializer implements CommandLineRunner {

    private final GerantRepository gerants;
    private final ClientRepository clients;
    private final FactureRepository factures;
    private final PasswordEncoder encoder;

    @Override
    public void run(String... args) {
        if (gerants.count() == 0) {
            gerants.save(new Gerant(null, "Administrateur", "admin", encoder.encode("admin123")));
        }
        if (clients.count() == 0) {
            Client a = clients.save(new Client(null, "Moussa Diop", "771234567", "moussa",
      encoder.encode("client123")));
            Client b = clients.save(new Client(null, "Awa Ndiaye", "781112233", "awa",
      encoder.encode("client123")));
            Client c = clients.save(new Client(null, "Cheikh Fall", "765554433", "cheikh",
      encoder.encode("client123")));
            factures.save(new Facture(null, 150000.0, "Achat de materiel informatique", a.getId(),
      LocalDate.now().minusDays(12)));
            factures.save(new Facture(null, 45000.0, "Maintenance reseau", a.getId(),
      LocalDate.now().minusDays(5)));
            factures.save(new Facture(null, 98500.0, "Licence logicielle annuelle", b.getId(),
      LocalDate.now().minusDays(8)));
            factures.save(new Facture(null, 27500.0, "Formation utilisateurs", c.getId(),
      LocalDate.now().minusDays(2)));
        }
    }
}
```

15. backend/src/main/java/com/facturation/config/GlobalExceptionHandler.java (backend)

```java
package com.facturation.config;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.Map;
import java.util.stream.Collectors;

/** Renvoie les erreurs de validation sous la forme {"message": "..."}. */
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> validation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
                .map(e -> e.getDefaultMessage())
                .collect(Collectors.joining(" ; "));
        return ResponseEntity.badRequest().body(Map.of("message", msg));
    }
}
```

16. backend/src/main/java/com/facturation/config/SecurityConfig.java (backend)

```java
package com.facturation.config;

import com.facturation.security.JwtFilter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration
public class SecurityConfig {

    @Value("${app.cors.allowed-origin}")
    private String allowedOrigin;

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, JwtFilter jwtFilter) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .cors(Customizer.withDefaults())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/factures/mes-factures").hasRole("CLIENT")
                .requestMatchers("/api/**").hasRole("GERANT")
                .anyRequest().denyAll())
            .exceptionHandling(e -> e
                .authenticationEntryPoint((req, res, ex) -> res.sendError(401, "Non authentifie"))
                .accessDeniedHandler((req, res, ex) -> res.sendError(403, "Acces refuse")))
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration cfg = new CorsConfiguration();
        cfg.setAllowedOrigins(List.of(allowedOrigin));
        cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        cfg.setAllowedHeaders(List.of("Authorization", "Content-Type"));
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", cfg);
        return source;
    }
}
```

17. backend/src/main/java/com/facturation/controller/AuthController.java (backend)

```java
package com.facturation.controller;

import com.facturation.repository.ClientRepository;
import com.facturation.repository.GerantRepository;
import com.facturation.security.JwtService;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import lombok.RequiredArgsConstructor;
```

```java
import org.springframework.http.HttpStatus;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    public record LoginRequest(@NotBlank(message = "Login obligatoire") String login,
                               @NotBlank(message = "Mot de passe obligatoire") String password) {}

    public record LoginResponse(String token, String role, String nom, String userId) {}

    private final GerantRepository gerants;
    private final ClientRepository clients;
    private final PasswordEncoder encoder;
    private final JwtService jwt;

    @PostMapping("/login")
    public LoginResponse login(@Valid @RequestBody LoginRequest req) {
        var gerant = gerants.findByLogin(req.login());
        if (gerant.isPresent() && encoder.matches(req.password(), gerant.get().getPassword())) {
            var g = gerant.get();
            return new LoginResponse(jwt.generate(g.getLogin(), "GERANT", g.getId()), "GERANT",
      g.getNom(), g.getId());
        }
        var client = clients.findByLogin(req.login());
        if (client.isPresent() && encoder.matches(req.password(), client.get().getPassword())) {
            var c = client.get();
            return new LoginResponse(jwt.generate(c.getLogin(), "CLIENT", c.getId()), "CLIENT",
      c.getNom(), c.getId());
        }
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Login ou mot de passe
      incorrect");
    }
}
```

18. backend/src/main/java/com/facturation/controller/ClientController.java (backend)

```java
package com.facturation.controller;

import com.facturation.model.Client;
import com.facturation.repository.ClientRepository;
import com.facturation.repository.FactureRepository;
import com.facturation.repository.GerantRepository;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

import java.util.List;

/** CRUD + recherche des clients (reserve au gerant). */
@RestController
@RequestMapping("/api/clients")
@RequiredArgsConstructor
public class ClientController {

    private final ClientRepository clients;
    private final FactureRepository factures;
    private final GerantRepository gerants;
    private final PasswordEncoder encoder;

    @GetMapping
    public List<Client> list(@RequestParam(defaultValue = "") String q) {
        String term = q.trim();
        return term.isEmpty() ? clients.findAll()
                : clients.findByNomContainingIgnoreCaseOrTelContaining(term, term);
    }

```

```
35      @GetMapping("/{id}")
36      public Client get(@PathVariable String id) {
37          return clients.findById(id).orElseThrow(this::notFound);
38      }
39
40      @PostMapping
41      public ResponseEntity<Client> create(@Valid @RequestBody Client c) {
42          if (c.getPassword() == null || c.getPassword().isBlank()) {
43              throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Le mot de passe est
          obligatoire");
44          }
45          checkLoginFree(c.getLogin(), null);
46          c.setId(null);
47          c.setPassword(encoder.encode(c.getPassword()));
48          return ResponseEntity.status(HttpStatus.CREATED).body(clients.save(c));
49      }
50
51      @PutMapping("/{id}")
52      public Client update(@PathVariable String id, @Valid @RequestBody Client data) {
53          Client existing = clients.findById(id).orElseThrow(this::notFound);
54          checkLoginFree(data.getLogin(), id);
55          existing.setNom(data.getNom());
56          existing.setTel(data.getTel());
57          existing.setLogin(data.getLogin());
58          if (data.getPassword() != null && !data.getPassword().isBlank()) {
59              existing.setPassword(encoder.encode(data.getPassword()));
60          }
61          return clients.save(existing);
62      }
63
64      @DeleteMapping("/{id}")
65      public ResponseEntity<Void> delete(@PathVariable String id) {
66          if (!clients.existsById(id)) throw notFound();
67          factures.deleteByIdClient(id);   // suppression en cascade des factures du client
68          clients.deleteById(id);
69          return ResponseEntity.noContent().build();
70      }
71
72      private void checkLoginFree(String login, String selfId) {
73          boolean usedByGerant = gerants.findByLogin(login).isPresent();
74          boolean usedByOther = clients.findByLogin(login)
75                  .filter(x -> !x.getId().equals(selfId)).isPresent();
76          if (usedByGerant || usedByOther) {
77              throw new ResponseStatusException(HttpStatus.CONFLICT, "Ce login est deja utilise");
78          }
79      }
80
81      private ResponseStatusException notFound() {
82          return new ResponseStatusException(HttpStatus.NOT_FOUND, "Client introuvable");
83      }
84  }
```

19. backend/src/main/java/com/facturation/controller/FactureController.java (backend)

```
1   package com.facturation.controller;
2
3   import com.facturation.model.Client;
4   import com.facturation.model.Facture;
5   import com.facturation.model.FactureView;
6   import com.facturation.repository.ClientRepository;
7   import com.facturation.repository.FactureRepository;
8   import jakarta.validation.Valid;
9   import lombok.RequiredArgsConstructor;
10  import org.springframework.http.HttpStatus;
11  import org.springframework.http.ResponseEntity;
12  import org.springframework.security.core.Authentication;
13  import org.springframework.web.bind.annotation.*;
14  import org.springframework.web.server.ResponseStatusException;
15
16  import java.time.LocalDate;
17  import java.util.List;
18  import java.util.Map;
19  import java.util.stream.Collectors;
20
21  /** CRUD + recherche des factures (gerant) et consultation par le client. */
```

```java
@RestController
@RequestMapping("/api/factures")
@RequiredArgsConstructor
public class FactureController {

    private final FactureRepository factures;
    private final ClientRepository clients;

    private Map<String, String> clientNames() {
        return clients.findAll().stream().collect(Collectors.toMap(Client::getId, Client::getNom));
    }

    private FactureView view(Facture f, Map<String, String> names) {
        return new FactureView(f.getId(), f.getMontant(), f.getObjet(), f.getIdClient(),
                names.getOrDefault(f.getIdClient(), "(client supprime)"), f.getDate());
    }

    /** Gerant : toutes les factures, avec recherche par objet, client ou numero. */
    @GetMapping
    public List<FactureView> list(@RequestParam(defaultValue = "") String q) {
        Map<String, String> names = clientNames();
        String t = q.trim().toLowerCase();
        return factures.findAll().stream()
                .map(f -> view(f, names))
                .filter(v -> t.isEmpty()
                        || v.objet().toLowerCase().contains(t)
                        || v.clientNom().toLowerCase().contains(t)
                        || v.id().toLowerCase().contains(t))
                .toList();
    }

    /** Client connecte : uniquement ses propres factures (id lu dans le JWT). */
    @GetMapping("/mes-factures")
    public List<FactureView> mine(Authentication auth) {
        String clientId = auth.getName();
        Map<String, String> names = clientNames();
        return factures.findByIdClient(clientId).stream().map(f -> view(f, names)).toList();
    }

    @GetMapping("/{id}")
    public FactureView get(@PathVariable String id) {
        Facture f = factures.findById(id).orElseThrow(this::notFound);
        return view(f, clientNames());
    }

    @PostMapping
    public ResponseEntity<FactureView> create(@Valid @RequestBody Facture f) {
        Client c = clients.findById(f.getIdClient())
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.BAD_REQUEST, "Client
  introuvable"));
        f.setId(null);
        if (f.getDate() == null) f.setDate(LocalDate.now());
        Facture saved = factures.save(f);
        return ResponseEntity.status(HttpStatus.CREATED).body(view(saved, Map.of(c.getId(),
  c.getNom())));
    }

    @PutMapping("/{id}")
    public FactureView update(@PathVariable String id, @Valid @RequestBody Facture data) {
        Facture existing = factures.findById(id).orElseThrow(this::notFound);
        Client c = clients.findById(data.getIdClient())
                .orElseThrow(() -> new ResponseStatusException(HttpStatus.BAD_REQUEST, "Client
  introuvable"));
        existing.setMontant(data.getMontant());
        existing.setObjet(data.getObjet());
        existing.setIdClient(data.getIdClient());
        if (data.getDate() != null) existing.setDate(data.getDate());
        return view(factures.save(existing), Map.of(c.getId(), c.getNom()));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable String id) {
        if (!factures.existsById(id)) throw notFound();
        factures.deleteById(id);
        return ResponseEntity.noContent().build();
    }

```

```
    private ResponseStatusException notFound() {
        return new ResponseStatusException(HttpStatus.NOT_FOUND, "Facture introuvable");
    }
}
```

20. frontend/package.json (frontend)

```
{
  "name": "facturation-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.2"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.4.8"
  }
}
```

21. frontend/vite.config.js (frontend)

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: { port: 5173 },
});
```

22. frontend/index.html (frontend)

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Facturation - TP2 JEE</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

23. frontend/.env (frontend)

```
VITE_API_URL=http://localhost:8080/api
```

24. frontend/src/main.jsx (frontend)

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App.jsx";
import "./styles.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

25. frontend/src/api.js (frontend)

```
// Couche d'acces a l'API Spring Boot (JWT envoye dans l'en-tete Authorization)
const BASE = import.meta.env.VITE_API_URL || "http://localhost:8080/api";

export function getAuth() {
  try {
    return JSON.parse(localStorage.getItem("auth"));
  } catch {
    return null;
  }
}
export const saveAuth = (data) => localStorage.setItem("auth", JSON.stringify(data));
export const clearAuth = () => localStorage.removeItem("auth");

async function request(path, options = {}) {
  const auth = getAuth();
  const res = await fetch(BASE + path, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...(auth ? { Authorization: `Bearer ${auth.token}` } : {}),
    },
  });
  if (res.status === 401 && path !== "/auth/login") {
    clearAuth();
    window.location.href = "/login";
    throw new Error("Session expirée, veuillez vous reconnecter");
  }
  if (!res.ok) {
    let message = "Une erreur est survenue";
    try {
      const data = await res.json();
      message = data.message || message;
    } catch { /* corps vide */ }
    throw new Error(message);
  }
  return res.status === 204 ? null : res.json();
}

const json = (method, body) => ({ method, body: JSON.stringify(body) });

export const api = {
  login: (creds) => request("/auth/login", json("POST", creds)),
  clients: {
    list: (q = "") => request(`/clients?q=${encodeURIComponent(q)}`),
    create: (c) => request("/clients", json("POST", c)),
    update: (id, c) => request(`/clients/${id}`, json("PUT", c)),
    remove: (id) => request(`/clients/${id}`, { method: "DELETE" }),
  },
  factures: {
    list: (q = "") => request(`/factures?q=${encodeURIComponent(q)}`),
    create: (f) => request("/factures", json("POST", f)),
    update: (id, f) => request(`/factures/${id}`, json("PUT", f)),
    remove: (id) => request(`/factures/${id}`, { method: "DELETE" }),
    mine: () => request("/factures/mes-factures"),
  },
};

export const formatMoney = (n) =>
  new Intl.NumberFormat("fr-FR").format(n) + " FCFA";
```

26. frontend/src/App.jsx (frontend)

```
import { Navigate, Route, Routes } from "react-router-dom";
import { getAuth } from "./api";
import Layout from "./Layout.jsx";
import LoginPage from "./pages/LoginPage.jsx";
import ClientsPage from "./pages/ClientsPage.jsx";
import FacturesPage from "./pages/FacturesPage.jsx";
import MesFacturesPage from "./pages/MesFacturesPage.jsx";

// Route protegee : exige une session et le bon role
function Protected({ role, children }) {
```

```
  const auth = getAuth();
  if (!auth) return <Navigate to="/login" replace />;
  if (auth.role !== role) return <Navigate to="/" replace />;
  return <Layout>{children}</Layout>;
}

function Home() {
  const auth = getAuth();
  if (!auth) return <Navigate to="/login" replace />;
  return <Navigate to={auth.role === "GERANT" ? "/gerant/clients" : "/client/factures"} replace />;
}

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/login" element={<LoginPage />} />
      <Route path="/gerant/clients" element={<Protected role="GERANT"><ClientsPage /></Protected>}
      />
      <Route path="/gerant/factures" element={<Protected role="GERANT"><FacturesPage /></Protected>}
       />
      <Route path="/client/factures" element={<Protected role="CLIENT"><MesFacturesPage
      /></Protected>} />
      <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
  );
}
```

27. frontend/src/Layout.jsx (frontend)

```
import { NavLink, useNavigate } from "react-router-dom";
import { clearAuth, getAuth } from "./api";

export default function Layout({ children }) {
  const auth = getAuth();
  const navigate = useNavigate();

  const logout = () => {
    clearAuth();
    navigate("/login");
  };

  return (
    <>
      <header className="navbar">
        <div className="brand">Facturation</div>
        <nav>
          {auth.role === "GERANT" ? (
            <>
              <NavLink to="/gerant/clients">Clients</NavLink>
              <NavLink to="/gerant/factures">Factures</NavLink>
            </>
          ) : (
            <NavLink to="/client/factures">Mes factures</NavLink>
          )}
        </nav>
        <div className="user">
          <span>{auth.nom} ({auth.role === "GERANT" ? "Gérant" : "Client"})</span>
          <button className="btn small secondary" onClick={logout}>Déconnexion</button>
        </div>
      </header>
      <main className="container">{children}</main>
    </>
  );
}
```

28. frontend/src/pages/LoginPage.jsx (frontend)

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { api, saveAuth } from "../api";

export default function LoginPage() {
  const [login, setLogin] = useState("");
  const [password, setPassword] = useState("");
```

```
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const submit = async (e) => {
    e.preventDefault();
    try {
      const data = await api.login({ login, password });
      saveAuth(data);
      navigate(data.role === "GERANT" ? "/gerant/clients" : "/client/factures");
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <div className="login-wrap">
      <div className="card">
        <h1>Connexion</h1>
        <p className="muted">Gestion de la facturation - TP2 JEE</p>
        {error && <div className="error">{error}</div>}
        <form onSubmit={submit}>
          <div className="field full">
            <label>Login</label>
            <input value={login} onChange={(e) => setLogin(e.target.value)} autoFocus />
          </div>
          <div className="field full">
            <label>Mot de passe</label>
            <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} />
          </div>
          <button className="btn full-btn" type="submit">Se connecter</button>
        </form>
      </div>
    </div>
  );
}
```

29. frontend/src/pages/ClientsPage.jsx (frontend)

```
import { useEffect, useState } from "react";
import { api } from "../api";

const EMPTY = { id: null, nom: "", tel: "", login: "", password: "" };

export default function ClientsPage() {
  const [clients, setClients] = useState([]);
  const [q, setQ] = useState("");
  const [form, setForm] = useState(null); // null = formulaire masque
  const [error, setError] = useState("");

  const load = async (term = q) => {
    try {
      setClients(await api.clients.list(term));
      setError("");
    } catch (e) {
      setError(e.message);
    }
  };

  useEffect(() => { load(""); }, []);

  const change = (field) => (e) => setForm({ ...form, [field]: e.target.value });

  const save = async (e) => {
    e.preventDefault();
    try {
      if (form.id) await api.clients.update(form.id, form);
      else await api.clients.create(form);
      setForm(null);
      load();
    } catch (err) {
      setError(err.message);
    }
  };

  const remove = async (c) => {
    if (!window.confirm(`Supprimer ${c.nom} et toutes ses factures ?`)) return;
```

```
    try {
      await api.clients.remove(c.id);
      load();
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <>
      <h1>Gestion des clients</h1>
      {error && <div className="error">{error}</div>}

      {form && (
        <div className="card">
          <h2>{form.id ? "Modifier le client" : "Nouveau client"}</h2>
          <form onSubmit={save}>
            <div className="field half"><label>Nom</label><input value={form.nom}
      onChange={change("nom")} /></div>
            <div className="field half"><label>Téléphone</label><input value={form.tel}
      onChange={change("tel")} /></div>
            <div className="field half"><label>Login (accès client)</label><input value={form.login}
       onChange={change("login")} /></div>
            <div className="field half">
              <label>Mot de passe {form.id && "(vide = inchangé)"}</label>
              <input type="password" value={form.password} onChange={change("password")} />
            </div>
            <button className="btn" type="submit">Enregistrer</button>{" "}
            <button className="btn secondary" type="button" onClick={() =>
      setForm(null)}>Annuler</button>
          </form>
        </div>
      )}

      <div className="card">
        <div className="toolbar">
          <form onSubmit={(e) => { e.preventDefault(); load(); }}>
            <input placeholder="Rechercher par nom ou téléphone..." value={q} onChange={(e) =>
      setQ(e.target.value)} />
          </form>
          <button className="btn" onClick={() => setForm({ ...EMPTY })}>+ Nouveau client</button>
        </div>
        <table>
          <thead>
            <tr><th>Nom</th><th>Téléphone</th><th>Login</th><th style={{ width: 170
      }}>Actions</th></tr>
          </thead>
          <tbody>
            {clients.length === 0 && <tr><td colSpan="4" className="muted">Aucun client
      trouvé.</td></tr>}
            {clients.map((c) => (
              <tr key={c.id}>
                <td>{c.nom}</td>
                <td>{c.tel}</td>
                <td>{c.login}</td>
                <td>
                  <button className="btn small" onClick={() => setForm({ ...c, password: ""
      })}>Modifier</button>{" "}
                  <button className="btn small danger" onClick={() => remove(c)}>Supprimer</button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </>
  );
}
```

30. frontend/src/pages/FacturesPage.jsx (frontend)

```
import { useEffect, useState } from "react";
import { api, formatMoney } from "../api";

const EMPTY = { id: null, montant: "", objet: "", idClient: "" };

```

```jsx
export default function FacturesPage() {
  const [factures, setFactures] = useState([]);
  const [clients, setClients] = useState([]);
  const [q, setQ] = useState("");
  const [form, setForm] = useState(null);
  const [error, setError] = useState("");

  const load = async (term = q) => {
    try {
      setFactures(await api.factures.list(term));
      setError("");
    } catch (e) {
      setError(e.message);
    }
  };

  useEffect(() => {
    load("");
    api.clients.list("").then(setClients).catch((e) => setError(e.message));
  }, []);

  const change = (field) => (e) => setForm({ ...form, [field]: e.target.value });

  const save = async (e) => {
    e.preventDefault();
    const payload = { ...form, montant: parseFloat(form.montant) };
    try {
      if (form.id) await api.factures.update(form.id, payload);
      else await api.factures.create(payload);
      setForm(null);
      load();
    } catch (err) {
      setError(err.message);
    }
  };

  const remove = async (f) => {
    if (!window.confirm(`Supprimer la facture "${f.objet}" ?`)) return;
    try {
      await api.factures.remove(f.id);
      load();
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <>
      <h1>Gestion des factures</h1>
      {error && <div className="error">{error}</div>}

      {form && (
        <div className="card">
          <h2>{form.id ? "Modifier la facture" : "Nouvelle facture"}</h2>
          <form onSubmit={save}>
            <div className="field half">
              <label>Client</label>
              <select value={form.idClient} onChange={change("idClient")}>
                <option value="">-- Choisir un client --</option>
                {clients.map((c) => <option key={c.id} value={c.id}>{c.nom}</option>)}
              </select>
            </div>
            <div className="field half">
              <label>Montant (FCFA)</label>
              <input type="number" min="0" step="any" value={form.montant}
      onChange={change("montant")} />
            </div>
            <div className="field full"><label>Objet</label><input value={form.objet}
      onChange={change("objet")} /></div>
            <button className="btn" type="submit">Enregistrer</button>{" "}
            <button className="btn secondary" type="button" onClick={() =>
      setForm(null)}>Annuler</button>
          </form>
        </div>
      )}

      <div className="card">
```

```
        <div className="toolbar">
          <form onSubmit={(e) => { e.preventDefault(); load(); }}>
            <input placeholder="Rechercher par objet, client ou numéro..." value={q} onChange={(e) => setQ(e.target.value)} />
          </form>
          <button className="btn" onClick={() => setForm({ ...EMPTY })}>+ Nouvelle facture</button>
        </div>
        <table>
          <thead>
            <tr><th>Date</th><th>Client</th><th>Objet</th><th>Montant</th><th style={{ width: 170 }}>Actions</th></tr>
          </thead>
          <tbody>
            {factures.length === 0 && <tr><td colSpan="5" className="muted">Aucune facture trouvée.</td></tr>}
            {factures.map((f) => (
              <tr key={f.id}>
                <td>{f.date}</td>
                <td>{f.clientNom}</td>
                <td>{f.objet}</td>
                <td className="num">{formatMoney(f.montant)}</td>
                <td>
                  <button className="btn small" onClick={() => setForm({ ...f })}>Modifier</button>{" "}
                  <button className="btn small danger" onClick={() => remove(f)}>Supprimer</button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </>
  );
}
```

31. frontend/src/pages/MesFacturesPage.jsx (frontend)

```
import { useEffect, useState } from "react";
import { api, formatMoney, getAuth } from "../api";

export default function MesFacturesPage() {
  const [factures, setFactures] = useState([]);
  const [error, setError] = useState("");
  const auth = getAuth();

  useEffect(() => {
    api.factures.mine().then(setFactures).catch((e) => setError(e.message));
  }, []);

  const total = factures.reduce((sum, f) => sum + f.montant, 0);

  return (
    <>
      <h1>Bonjour {auth.nom}, voici vos factures</h1>
      {error && <div className="error">{error}</div>}
      <div className="card">
        <table>
          <thead>
            <tr><th>N°</th><th>Date</th><th>Objet</th><th>Montant</th></tr>
          </thead>
          <tbody>
            {factures.length === 0 && <tr><td colSpan="4" className="muted">Vous n'avez aucune facture.</td></tr>}
            {factures.map((f) => (
              <tr key={f.id}>
                <td>{f.id.slice(-6).toUpperCase()}</td>
                <td>{f.date}</td>
                <td>{f.objet}</td>
                <td className="num">{formatMoney(f.montant)}</td>
              </tr>
            ))}
          </tbody>
        </table>
        <div className="total">Total : <strong>{formatMoney(total)}</strong></div>
      </div>
    </>
```

```
  );
}
```

32. frontend/src/styles.css (frontend)

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, Helvetica, sans-serif; background: #f1f5f9; color: #1e293b; }
h1 { font-size: 24px; margin: 0 0 18px; }
h2 { font-size: 18px; margin: 0 0 14px; }
.muted { color: #64748b; font-size: 14px; }

.navbar { background: #1e3a8a; color: #fff; padding: 14px 28px; overflow: hidden; }
.navbar .brand { float: left; font-size: 20px; font-weight: bold; padding-top: 4px; }
.navbar nav { float: left; margin-left: 30px; }
.navbar nav a { color: #cbd5e1; text-decoration: none; margin-right: 8px; padding: 8px 14px; border-radius: 6px; display: inline-block; }
.navbar nav a.active { background: #2563eb; color: #fff; }
.navbar .user { float: right; padding-top: 3px; font-size: 14px; }
.navbar .user span { margin-right: 12px; }

.container { max-width: 1000px; margin: 28px auto; padding: 0 20px; }
.card { background: #fff; border-radius: 10px; padding: 22px; box-shadow: 0 1px 4px rgba(0,0,0,.12); margin-bottom: 22px; }

input, select { padding: 9px 11px; border: 1px solid #cbd5e1; border-radius: 6px; font-size: 14px; width: 100%; background: #fff; }
.field { margin-bottom: 14px; vertical-align: top; display: inline-block; }
.field.full { display: block; }
.field.half { width: 49%; padding-right: 12px; }
.field label { display: block; font-size: 13px; font-weight: bold; margin-bottom: 5px; }

.btn { background: #2563eb; color: #fff; border: 0; padding: 10px 18px; border-radius: 6px; cursor: pointer; font-size: 14px; }
.btn.secondary { background: #64748b; }
.btn.danger { background: #dc2626; }
.btn.small { padding: 6px 11px; font-size: 13px; }
.full-btn { width: 100%; }

.toolbar { overflow: hidden; margin-bottom: 16px; }
.toolbar form { float: left; width: 62%; }
.toolbar > .btn { float: right; }

table { width: 100%; border-collapse: collapse; }
th { background: #e2e8f0; text-align: left; padding: 10px; font-size: 13px; }
td { padding: 10px; border-bottom: 1px solid #e2e8f0; font-size: 14px; }
td.num { white-space: nowrap; font-weight: bold; }
.total { text-align: right; margin-top: 16px; font-size: 16px; }

.error { background: #fee2e2; color: #991b1b; padding: 10px 14px; border-radius: 6px; margin-bottom: 16px; font-size: 14px; }
.login-wrap { width: 400px; margin: 90px auto; }
```